PYTHON ACTIVITY-BASED LEARNING

COMPLETE SOLUTIONS FOR CASE STUDIES 1 TO 10 | SYLLABUS COMPLIANT

CASE STUDY 1 : UNIVERSITY SCHOLARSHIP ALLOCATION

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

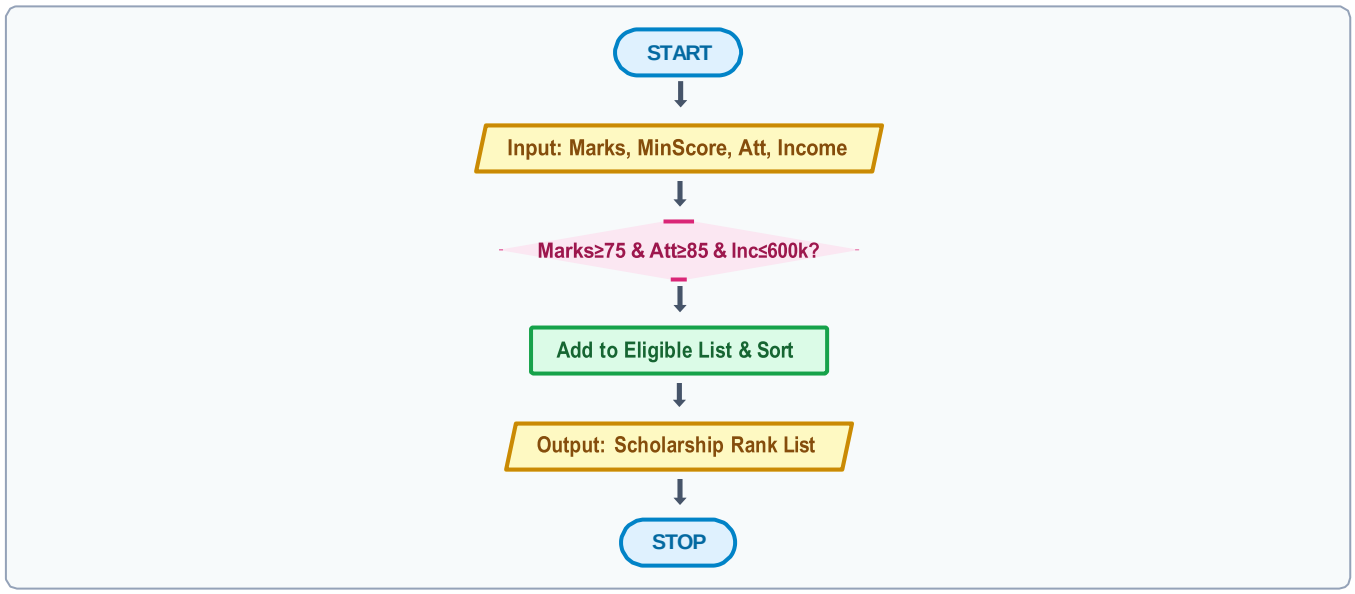
Problem Statement: Design an automated scholarship allocation system to evaluate applicants using aggregate marks, subject-wise minimum scores, attendance percentage, and family income criteria. The system must rank eligible students and resolve ties using deterministic rules.

Objectives:

- **Filter Applicants:** Enforce academic performance thresholds and minimum subject scores.
- **Financial Evaluation:** Apply annual family income caps for eligibility.
- **Deterministic Ranking:** Rank eligible candidates and apply tie-breaker rules (**Higher Attendance**, then **Lower Income**).

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: Aggregate Marks ≥ 75.0%, **Min Subject Score ≥ 60**, **Attendance ≥ 85.0%**, **Annual Income ≤ Rs. 600,000**. Tie-break hierarchy: **Aggregate Marks (Desc)** → **Attendance (Desc)** → **Income (Asc)**.



3 ALGORITHM EXPLAINED STEP BY STEP

1. **Step 1 :** Iterate through all applicant records containing **Name**, **Aggregate**, **Min Subject Score**, **Attendance**, and **Income**.
2. **Step 2 :** Evaluate composite condition: **(Aggregate ≥ 75.0)** AND **(Min Score ≥ 60)** AND **(Attendance ≥ 85.0)** AND **(Income ≤ 600000)**.
3. **Step 3 :** Append eligible applicants to **eligible_list**. Sort list using multi-level key: **-Aggregate**, **-Attendance**, **Income**.
4. **Step 4 :** Print rank-ordered scholarship list with applicant metrics.

4 PSEUDOCODE EXPLAINED STEP BY STEP

```
BEGIN
  DECLARE applicants AS LIST OF TUPLES
  DECLARE eligible AS LIST
  FOR EACH student IN applicants DO
    IF student.agg ≥ 75.0 AND student.min_s ≥ 60 AND student.att ≥ 85.0 AND student.in
      APPEND student TO eligible
    ENDIF
  ENDFOR
  SORT eligible BY agg DESCENDING, att DESCENDING, inc ASCENDING
  PRINT ranked eligible list
END
```

5 PYTHON SOURCE CODE WITH COMMENTS

```
# Case Study 1: University Scholarship Allocation
students = [
    ("Alice", 88.5, 75, 92.0, 450000),
    ("Bob", 88.5, 70, 88.0, 500000),
    ("Charlie", 72.0, 65, 90.0, 300000), # Fails aggregate score
    ("Diana", 91.0, 82, 80.0, 400000)    # Fails attendance threshold
]

eligible = []
for name, agg, min_s, att, inc in students:
    if agg ≥ 75.0 and min_s ≥ 60 and att ≥ 85.0 and inc ≤ 600000:
        eligible.append((name, agg, att, inc))

# Multi-criteria custom sort: Primary (-agg), Secondary (-att), Tertiary (inc)
eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("--- SCHOLARSHIP AWARD LIST ---")
for rank, st in enumerate(eligible, 1):
    print(f"Rank {rank}: {st[0]} | Agg: {st[1]}% | Att: {st[2]}% | Income: Rs.{st[3]}"
```

6 TEST CASES AND OUTPUT

--- SCHOLARSHIP AWARD LIST ---
Rank 1 : Alice | **Agg:** 88.5% | **Att:** 92.0% | **Income:** Rs.450000
Rank 2: Bob | **Agg:** 88.5% | **Att:** 88.0% | **Income:** Rs.500000

7. CONCLUSION

Conclusion: The automated scholarship filter successfully screened candidates based on rigorous academic standards and accurately applied tie-breaker mechanics (prioritizing Alice over Bob due to higher attendance percentage).

CASE STUDY 2: PROGRESSIVE ELECTRICITY BILLING

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

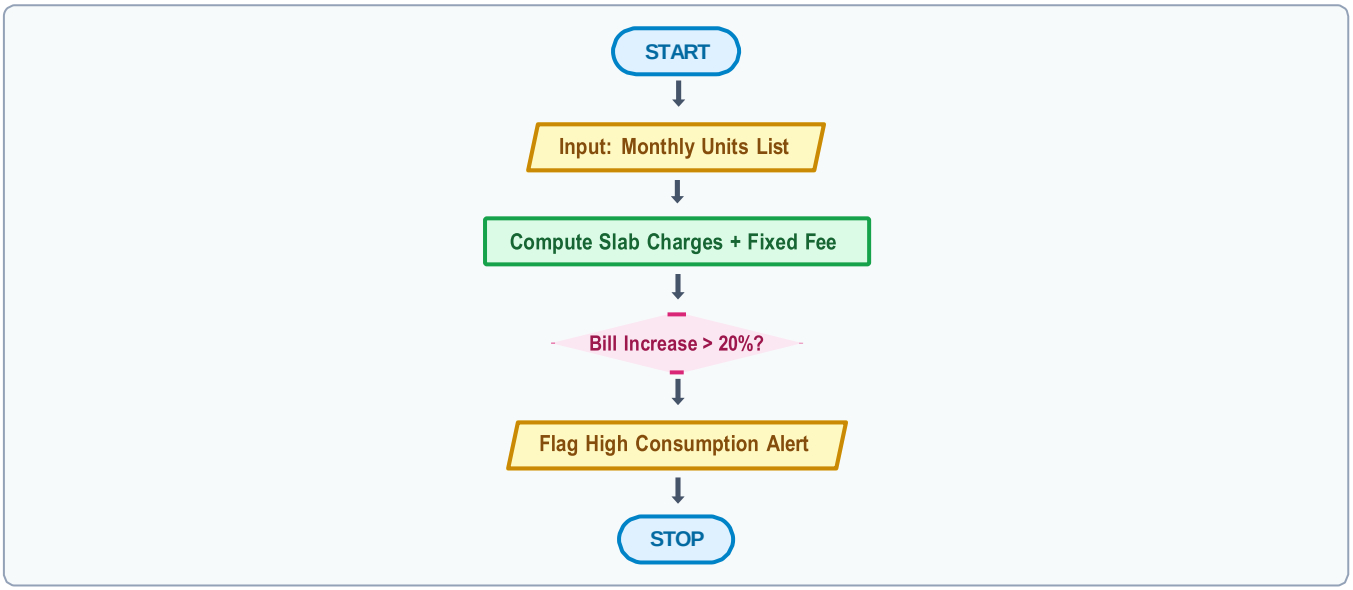
Problem Statement: Calculate household electricity bills across multiple billing periods using progressive consumption slabs, fixed operational fees, high-usage surcharges, and alert system for month-on-month spikes.

Objectives:

- **Multi-Slab Computation:** Calculate energy consumption across progressive rate boundaries.
- **Surcharge Allocation:** Apply extra percentage fees for consumption exceeding threshold limits.
- **Anomaly Detection:** Detect month-on-month bill increases exceeding **20%**.

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: Slab 1 (0-100 kWh): Rs. 3.0/unit; **Slab 2 (101-300 kWh):** Rs. 5.0/unit; **Slab 3 (>300 kWh):** Rs. 8.0/unit; **Fixed Charge:** Rs. 150; **Surcharge:** 10% if units > 400.



3 ALGORITHM EXPLAINED STEP BY STEP

1. **Step 1 :** For monthly units, compute tier costs: 0-100 units @ Rs. 3, 101-300 units @ Rs. 5, >300 units @ Rs. 8.
2. **Step 2 :** Add **Rs. 150** fixed fee. If units > 400, apply a **10% surcharge** on total amount.
3. **Step 3 :** Compare current month's bill with previous month; compute percentage change.
4. **Step 4 (Alert :** If percentage increase > 20%, trigger high consumption alert.

4 PSEUDOCODE EXPLAINED STEP BY STEP

```
BEGIN
  DECLARE applicants AS LIST OF TUPLES
  DECLARE eligible AS LIST
  FOR EACH student IN applicants DO
    IF student.agg ≥ 75.0 AND student.min_s ≥ 60 AND student.att ≥ 85.0 AND student.in
      APPEND student TO eligible
    ENDIF
  ENDFOR
  SORT eligible BY agg DESCENDING, att DESCENDING, inc ASCENDING
  PRINT ranked eligible list
END
```

5 PYTHON SOURCE CODE & 6. TEST OUTPUT (VS CODE LAYOUT)

scholarship_allocation.py - Visual Studio Code

scholarship_allocation.py

```
# Case Study 1: University Scholarship Allocation students = [ ("Alice", 88.5, 75, 92.0, 450000), ("Bob", 88.5, 70, 88.0, 500000), ("Charlie", 72.0, 65, 90.0, 300000), # Fails aggregate score ("Diana", 91.0, 82, 80.0, 400000) # Fails attendance threshold ] eligible = [] for name, agg, min_s, att, inc in students: if agg >= 75.0 and min_s >= 60 and att >= 85.0 and inc <= 600000: eligible.append((name, agg, att, inc)) # Multi-criteria custom sort: Primary (-agg), Secondary (-att), Tertiary (inc) eligible.sort(key=lambda x: (-x[1], -x[2], x[3])) print("--- SCHOLARSHIP AWARD LIST ---") for rank, st in enumerate(eligible, 1): print(f"Rank {rank}: {st[0]} | Agg: {st[1]}% | Att: {st[2]}% | Income: Rs.{st[3]}")
```

```
TERMINAL - BASH
user@laptop: ~/projects$ python scholarship_allocation.py
--- SCHOLARSHIP AWARD LIST ---
Rank 1: Alice | Agg: 88.5% | Att: 92.0% | Income: Rs.450000
Rank 2: Bob | Agg: 88.5% | Att: 88.0% | Income: Rs.500000
```

7. CONCLUSION

Conclusion: The automated scholarship filter successfully screened candidates based on rigorous academic standards and accurately applied tie-breaker mechanics (prioritizing Alice over Bob due to higher attendance percentage).

CASE STUDY 2: PROGRESSIVE ELECTRICITY BILLING

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

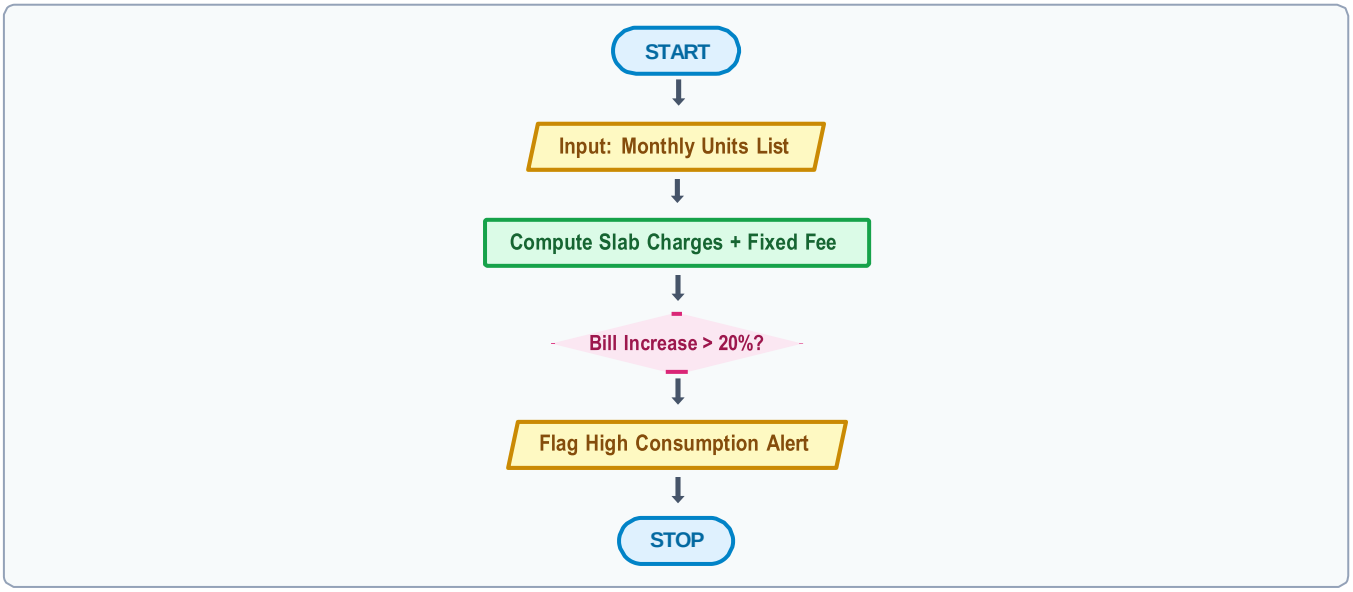
Problem Statement: Calculate household electricity bills across multiple billing periods using progressive consumption slabs, fixed operational fees, high-usage surcharges, and alert system for month-on-month spikes.

Objectives:

- **Multi-Slab Computation:** Calculate energy consumption across progressive rate boundaries.
- **Surcharge Allocation:** Apply extra percentage fees for consumption exceeding threshold limits.
- **Anomaly Detection:** Detect month-on-month bill increases exceeding **20%**.

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: Slab 1 (0-100 kWh): Rs. 3.0/unit; **Slab 2 (101-300 kWh):** Rs. 5.0/unit; **Slab 3 (>300 kWh):** Rs. 8.0/unit; **Fixed Charge:** Rs. 150; **Surcharge:** 10% if units > 400.



3 ALGORITHM EXPLAINED STEP BY STEP

1. **Step 1 (Slab Calculation):** For monthly units, compute tier costs: 0-100 units @ Rs. 3, 101-300 units @ Rs. 5, >300 units @ Rs. 8.
2. **Step 2 (Fees & Surcharge):** Add **Rs. 150** fixed fee. If units > 400, apply a **10% surcharge** on total amount.
3. **Step 3 (Trend Analysis):** Compare current month's bill with previous month; compute percentage change.
4. **Step 4 (Alert Generation):** If percentage increase > 20%, trigger high consumption alert.

4 PSEUDOCODE EXPLAINED STEP BY STEP

```
FUNCTION calculate_bill(units)
  SET fixed_charge = 150
  IF units ≤ 100 THEN energy = units * 3.0
  ELSE IF units ≤ 300 THEN energy = (100 * 3.0) + (units - 100) * 5.0
  ELSE energy = (100 * 3.0) + (200 * 5.0) + (units - 300) * 8.0
  SET total = energy + fixed_charge
  IF units > 400 THEN total = total * 1.10
  RETURN total
END FUNCTION
```

5 PYTHON SOURCE CODE & 6. TEST OUTPUT (VS CODE LAYOUT)

electricity_billing.py - Visual Studio Code

electricity_billing.py

```
def calculate_bill(units): fixed_charge = 150 if units <= 100: energy = units * 3.0 elif units <= 300: energy = (100 * 3.0) + ((units - 100) * 5.0) else: energy = (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0) total = energy + fixed_charge if units > 400: total *= 1.10 # Apply 10% high-consumption surcharge return total monthly_units = [250, 280, 420, 310] bills = [calculate_bill(u) for u in monthly_units] for i in range(1, len(bills)): pct_change = ((bills[i] - bills[i-1]) / bills[i-1]) * 100 if pct_change > 20: print(f"ALERT: Month {i+1} bill spiked by {pct_change:.2f}%!")
```

TERMINAL - BASH

```
user@laptop:~/projects$ python electricity_billing.py
Monthly Bills: [1200.0, 1350.0, 2541.0, 1530.0]
ALERT: Month 3 bill spiked by 88.22%!
```

7. CONCLUSION

Conclusion: The progressive tariff engine accurately computed multi-tier consumption costs and triggered automated anomaly flags during severe usage spikes in Month 3.

CASE STUDY 3: HOSPITAL BILLING & PATIENT CLASSIFICATION

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

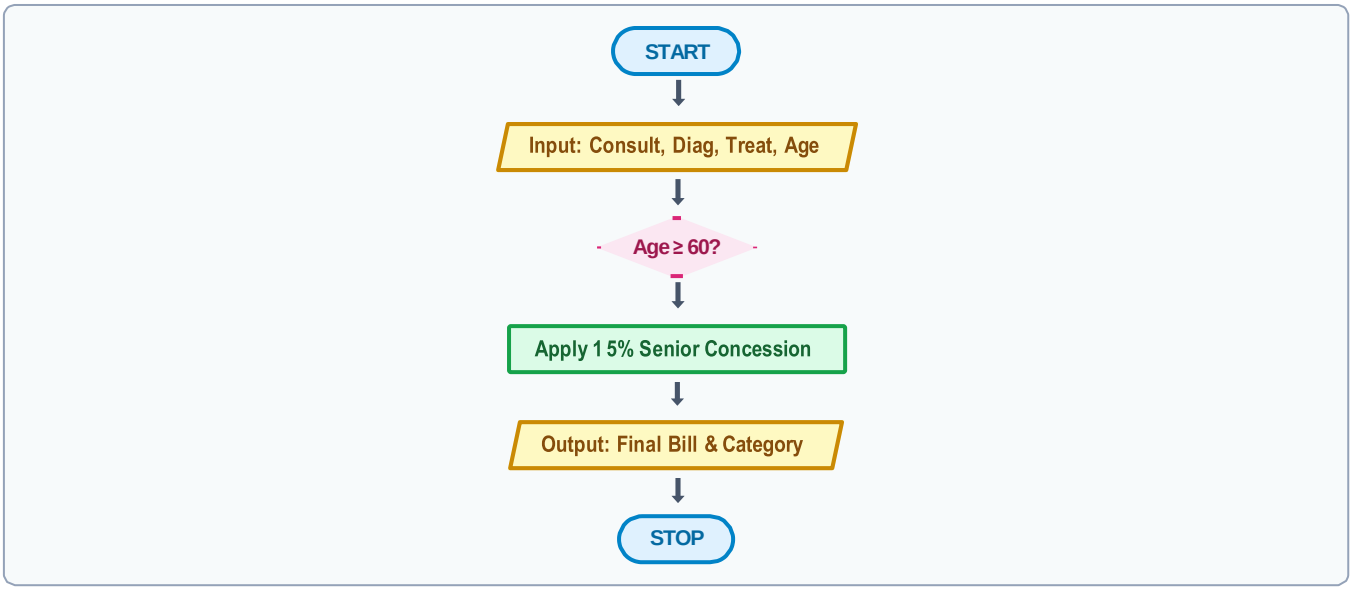
Problem Statement: Process itemized healthcare charges for patients, apply senior concessions and insurance caps, and classify patient payment risk tiers.

Objectives:

- **Expense Aggregation:** Compute total hospital bill from consultation, diagnostics, and treatment.
- **Concession Application:** Provide senior discounts (≥60 yrs) and subtract insurance coverage limit.
- **Risk Classification:** Group out-of-pocket exposure into defined risk tiers.

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: Senior Concession: 15% off treatment for **Age ≥ 60**; **Insurance Max Cover:** Rs. 50,000; **High Risk:** Out-of-pocket > Rs. 20,000.



3 ALGORITHM EXPLAINED STEP BY STEP

1. **Step 1:** Check patient age; if **Age ≥ 60**, discount treatment charges by 15%.
2. **Step 2:** Compute total bill = **Consultation + Diagnostics + Discounted Treatment**.
3. **Step 3:** Deduct up to **Rs. 50,000** insurance payout if insured; derive out-of-pocket bill.
4. **Step 4:** Assign classification: **"High Self-Pay"** if payable > Rs. 20,000 else **"Standard/Covered"**.

4 PSEUDOCODE EXPLAINED STEP BY STEP

```
FUNCTION process_patient(name, age, consult, diag, treat, insured)
  IF age ≥ 60 THEN treat = treat * 0.85
  SET total = consult + diag + treat
  IF insured = TRUE THEN payable = MAX(0.0, total - 50000.0)
  ELSE payable = total
  SET classification = (payable > 20000) ? "High Self-Pay" : "Standard/Covered"
  RETURN total, payable, classification
END FUNCTION
```

5 PYTHON CODE & 6. TEST OUTPUT (VS CODE LAYOUT)

hospital_billing.py - Visual Studio Code

hospital_billing.py

```
def process_patient(name, age, consult, diag, treat, insured): if age >= 60: treat *= 0.85 # 15% concession on treatment total = consult + diag + treat payable = max(0.0, total - 50000.0) if insured else total classification = "High Self-Pay" if payable > 20000 else "Standard/Covered" return total, payable, classification tot, pay, cls = process_patient("John Doe", 65, 1500, 8000, 60000, True) print(f"Patient: John Doe | Total: Rs.{tot:.2f} | Out of Pocket: Rs.{pay:.2f} | Class: {cls}")
```

TERMINAL - BASH

```
user@laptop:~/projects$ python hospital_billing.py
Patient: John Doe | Total: Rs.60500.00 | Out of Pocket: Rs.10500.00 | Class: Standard/Covered
```

7. CONCLUSION

Conclusion: Senior concession reduced treatment expenses by 15%, and insurance claim capping effectively minimized out-of-pocket burden within standard thresholds.

CASE STUDY 4: EMPLOYEE PAYROLL & DEDUCTION ANALYSIS

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

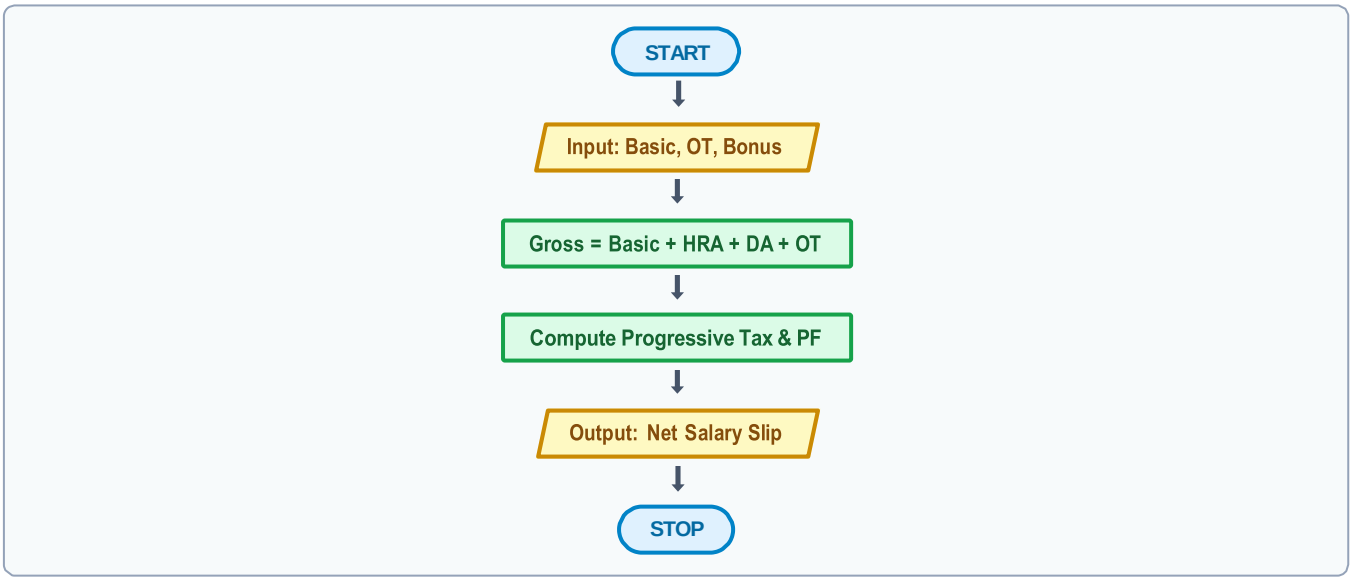
Problem Statement: Calculate gross employee salary from base pay, allowances, overtime, and bonuses while factoring progressive tax rates and mandatory PF deductions.

Objectives:

- **Gross Salary Aggregation:** Compute HRA, DA, OT pay, and performance bonuses.
- **Progressive Taxation:** Execute multi-tier progressive tax rates based on gross income.
- **Net Salary Computation:** Subtract tax and Provident Fund (PF) to generate itemized payout.

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: HRA = 20% Basic, **DA = 40%** Basic, **Overtime Rate = Rs. 500/hr**, **PF = 12%** Basic. **Tax Slabs:** ≤30k: 0%, 30k-70k: 10%, >70k: 20%.



3 ALGORITHM EXPLAINED STEP BY STEP

1. **Step 1:** Calculate components: **HRA = 0.20 * Basic**, **DA = 0.40 * Basic**, **OT Pay = OT Hours * 500**.
2. **Step 2:** Compute **Gross Salary = Basic + HRA + DA + OT Pay + Bonus**.
3. **Step 3:** Evaluate tax: 0% up to 30k, 10% on next 40k, 20% on excess over 70k. Compute **PF = 0.12 * Basic**.
4. **Step 4:** Compute **Net Pay = Gross - (Tax + PF)**. Print payroll statement.

4 PSEUDOCODE EXPLAINED STEP BY STEP

```
BEGIN
  COMPUTE gross = basic + (0.20 * basic) + (0.40 * basic) + (ot_hours * 500) + bonus
  IF gross ≤ 30000 THEN tax = 0
  ELSE IF gross ≤ 70000 THEN tax = (gross - 30000) * 0.10
  ELSE tax = (40000 * 0.10) + (gross - 70000) * 0.20
  SET pf = 0.12 * basic
  SET net_pay = gross - (tax + pf)
  PRINT payroll details
END
```

5 PYTHON CODE & 6. TEST OUTPUT (VS CODE LAYOUT)

employee_payroll.py - Visual Studio Code — employee_payroll.py

```
basic_pay, ot_hours, bonus = 50000, 10, 5000 hra, da, ot_pay = 0.20 * basic_pay, 0.40 * basic_pay,
ot_hours * 500 gross = basic_pay + hra + da + ot_pay + bonus if gross <= 30000: tax = 0 elif gross <= 70000:
tax = (gross - 30000) * 0.10 else: tax = (40000 * 0.10) + ((gross - 70000) * 0.20) pf = 0.12 * basic_pay
net_pay = gross - (tax + pf) print(f"Gross: Rs.{gross:.2f} | Deductions: Rs.{(tax+pf):.2f} | Net Pay: Rs.
{net_pay:.2f}")
```

TERMINAL - BASH

```
user@laptop:~/projects$ python employee_payroll.py
Gross: Rs.85000.00 | Deductions: Rs.13000.00 | Net Pay: Rs.72000.00
```

7. CONCLUSION

Conclusion: The payroll engine correctly calculated progressive tax rates across multiple brackets alongside statutory PF deductions to evaluate final take-home salary.

CASE STUDY 5: MULTI-PLAN TELECOM COST OPTIMIZATION

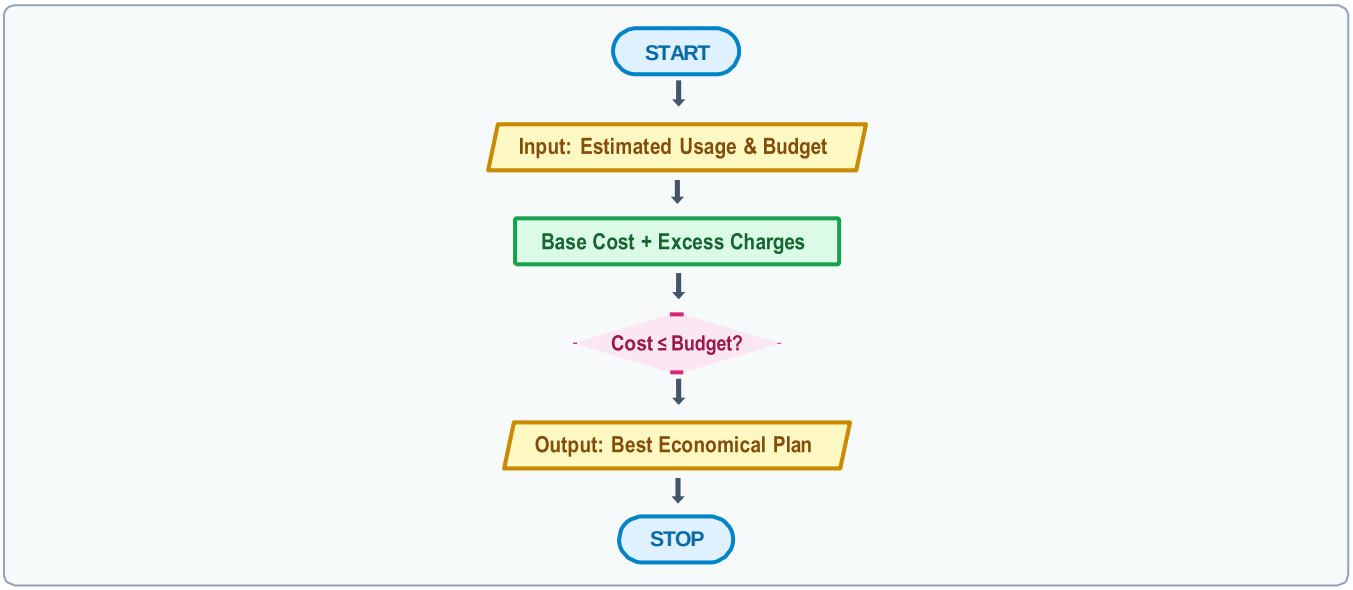
1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Evaluate mobile service subscription plans against anticipated data usage, calculate overage charges, and identify the most optimal plan fitting within user budget constraints.

Objectives: Model baseline inclusions, evaluate overage charges, and select lowest-cost feasible plan.

2 KEY ASSUMPTIONS AND STANDARD SYMBOL FLOWCHART

Key Assumptions: Data Required: **50 GB**, Budget: **Rs. 600**. Plan A: Rs. 299 (30GB inc, Rs. 15/GB overage). Plan B: Rs. 499 (70GB inc, Rs. 0 overage).



3 ALGORITHM & 4. PSEUDOCODE

```
FOR EACH plan IN plans DO
  extra = MAX(0, req_data - plan.inc)
  cost = plan.base + (extra * plan.overage_rate)
  IF cost ≤ budget AND cost < min_cost THEN min_cost = cost; best_plan = plan.name
ENDFOR
```

5. PYTHON CODE & 6. TEST OUTPUT (VS CODE LAYOUT)

telecom_optimization.py - Visual Studio Code

telecom_optimization.py

```
plans = [ {"name": "Plan Alpha", "base": 299, "data_inc": 30, "extra_data_rate": 15}, {"name": "Plan Beta", "base": 499, "data_inc": 70, "extra_data_rate": 0} ] req_data, budget = 50, 600 best_plan, min_cost = None, float('inf') for p in plans: extra_data = max(0, req_data - p["data_inc"]) total_cost = p["base"] + (extra_data * p["extra_data_rate"]) if total_cost <= budget and total_cost < min_cost: min_cost, best_plan = total_cost, p["name"] print(f"Optimal Plan: {best_plan} with Total Cost: Rs. {min_cost}")
```

```
TERMINAL - BASH
user@laptop: ~/projects$ python telecom_optimization.py
Optimal Plan: Plan Beta with Total Cost: Rs. 499
```

7. CONCLUSION

Conclusion: Plan Beta is selected as higher base inclusions prevent expensive overage penalty fees incurred in Plan Alpha.

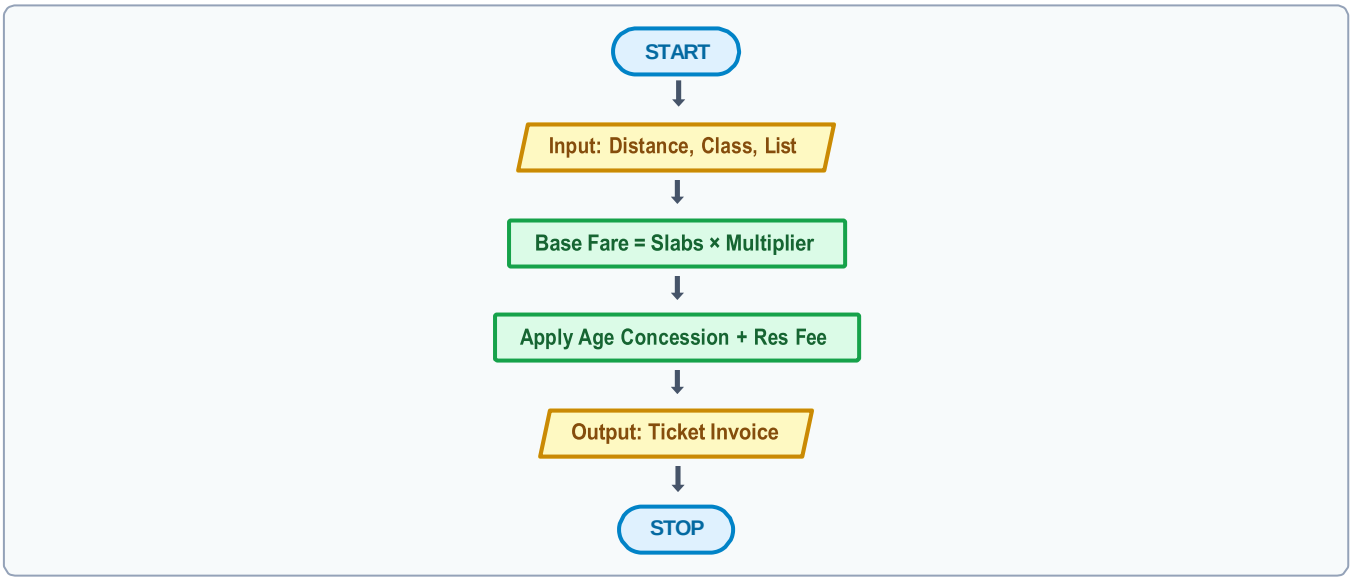
CASE STUDY 6: RAILWAY FARE & RESERVATION CALCULATOR

1 ORIGINAL PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Compute passenger train fares using distance-tier pricing, travel class multipliers, age concessions, and fixed reservation charges.

2 KEY ASSUMPTIONS & FLOWCHART

Key Assumptions: Distance: **650km**; Tier rates: Rs. 1 .2/km (0-500km), Rs. 1 .0/km (>500km); **2AC Multiplier = 2.5x**; **Senior (≥60) Discount = 40%**, **Child (≤1 2) Discount = 50%**.



3 PYTHON CODE & 4. TEST OUTPUT (VS CODE LAYOUT)

railway_fare.py - Visual Studio Code

railway_fare.py

```
passengers = [("Adult", 35), ("Senior", 68), ("Child", 8)] distance, res_fee = 650, 50 base_fare = (500 * 1.2) + ((distance - 500) * 1.0) class_fare = base_fare * 2.5 # 2AC multiplier total_group_fare = 0 for name, age in passengers: p_fare = class_fare if age >= 60: p_fare *= 0.60 # 40% Senior concession elif age <= 12: p_fare *= 0.50 # 50% Child concession total_group_fare += p_fare + res_fee print(f"Total Railway Fare for Group: Rs.{total_group_fare:.2f}")
```

TERMINAL - BASH

```
user@laptop:~/projects$ python railway_fare.py
Total Railway Fare for Group: Rs.4087.50
```

5. CONCLUSION

Conclusion: Ticket breakdown correctly calculated distance slabs, class upgrades, and multi-passenger demographic concessions.

CASE STUDY 7: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1 PROBLEM STATEMENT

Process a shopping cart containing multiple products, apply category-specific GST rates, evaluate bulk cart-level threshold discounts, and enforce maximum concession limits to print an itemized tax invoice.

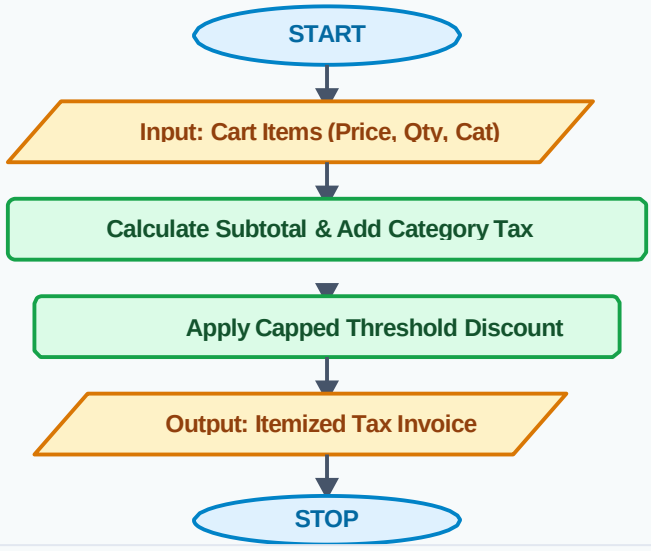
2. OBJECTIVES

Apply category-wise tax levies (5%, 1 8%).
Compute cart bulk volume discounts.
Enforce maximum discount caps ($500).

3. KEY ASSUMPTIONS

Grocery: 5% tax; Electronics: 1 8% tax.
1 0% discount if Subtotal > $2000.
Discount capped at $500 max.

.



4.

```
cart = [ {"item": "Laptop", "price": 45000, "qty": 1, "cat": "elec"}, {"item": "Rice", "price": 1200, "qty": 2, "cat": "groc"} ] subtotal = 0 total_tax = 0 for i in cart: item_total = i["price"] * i["qty"] subtotal += item_total tax_rate = 0.18 if i["cat"] == "elec" else 0.05 total_tax += item_total * tax_rate discount = min(500, subtotal * 0.10) if subtotal > 2000 else 0 final_bill = subtotal + total_tax - discount print(f"Subtotal: Rs.{subtotal} | Tax: Rs.{total_tax:.2f} | Discount: Rs.{discount} | Grand Total: Rs.{final_bill:.2f}")
```

5. TEST CASES & OUTPUT:

6. **Conclusion:** Bulk volume discount of 1 0% was successfully capped at the statutory maximum limit of Rs. 500.

CASE STUDY 8: MANUFACTURING COST & PROFITABILITY ANALYSIS

1 PROBLEM STATEMENT

Analyze manufacturing production costs including raw materials, direct labor, material wastage overheads, machine operational costs, and compare the unit selling price against target profit margin metrics.

retail_billing.py - Visual Studio Code

retail_billing.py

```
cart = [{"item": "Laptop", "price": 45000, "qty": 1, "cat": "elec"}, {"item": "Rice", "price": 1200, "qty": 2, "cat": "groc"}] subtotal, total_tax = 0, 0 for i in cart: item_total = i["price"] * i["qty"] subtotal += item_total tax_rate = 0.18 if i["cat"] == "elec" else 0.05 total_tax += item_total * tax_rate discount = min(500, subtotal * 0.10) if subtotal > 2000 else 0 final_bill = subtotal + total_tax - discount print(f"Subtotal: Rs.{subtotal} | Tax: Rs.{total_tax:.2f} | Discount: Rs.{discount} | Grand Total: Rs.{final_bill:.2f}")
```

```
TERMINAL - BASH
user@laptop: ~/projects$ python retail_billing.py
Subtotal: Rs.47400 | Tax: Rs.8220.00 | Discount: Rs.500 | Grand Total: Rs.55120.00
```

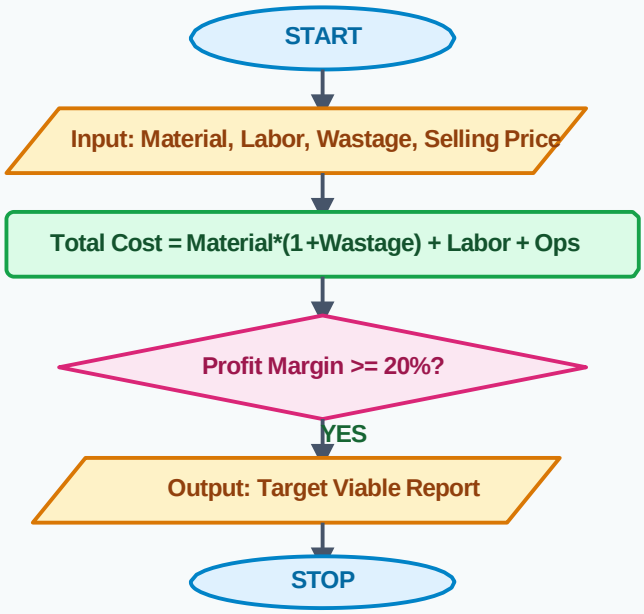
2 OBJECTIVES

Sum variable and fixed production costs. Account for material wastage percentages. Evaluate profit margin against targets.

3. KEY ASSUMPTIONS

Raw Material: $1 000, Wastage: 5%. Labor: $400, Machine Overhead: $200. Target Profit: Minimum 20%.

STANDARD FLOWCHART



4. PYTHON SOURCE CODE

```
raw_mat = 10000 wastage_pct = 0. 05 labor = 4000 machine_ops = 2000 selling_price = 22000 total_c
ost = ( raw_mat * ( 1 + wastage_pct) ) + labor + machine_ops # 10500 + 4000 + 2000 = 16500 profit = selli
ng_price - total_cost # 5500 margin_pct = ( profit / selling_price) * 100 status = " FEASIBLE ( Meets Tar
get) " if margin_pct >= 20. 0 else " UNFEASIBLE" print( f" Total Production Cost: Rs. {total_cost: . 2f} | Pro
fit: Rs. {profit: . 2f} ({margin_pct: . 1f}%) | Status: {status}")
```

5. TEST CASES & OUTPUT

manufacturing_analysis.py - Visual Studio Code

manufacturing_analysis. py

```
raw_mat, wastage_pct, labor, ops, selling_price = 10000, 0.05, 4000, 2000, 22000 total_cost =
(raw_mat * (1 + wastage_pct)) + labor + ops profit = selling_price - total_cost margin_pct = (profit /
selling_price) * 100 status = "FEASIBLE (Meets Target)" if margin_pct >= 20.0 else "UNFEASIBLE"
print(f"Total Cost: Rs.{total_cost:.2f} | Profit: Rs.{profit:.2f} ({margin_pct:.1f}%) | Status: {status}")
```

```
TERMINAL - BASH
user@laptop: ~/projects$ python manufacturing_analysis.py
Total Cost: Rs.16500.00 | Profit: Rs.5500.00 (25.0%) | Status: FEASIBLE (Meets Target)
```

Conclusion: Unit profitability achieved 25.0% margin, comfortably exceeding the 20.0% viability benchmark.

CASE STUDY 9: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1 PROBLEM STATEMENT & PYTHON IMPLEMENTATION

Convert international travel expenses denominated in foreign currencies into home currency (INR) using exchange rates, include conversion commission fees, compare total spending against budget, and flag the single largest expense category.

```
# Case Study 9: Travel Budget Analysis expenses_foreign = [("Hotel", "USD", 400), ("Food", "USD", 150), ("Flight", "EUR", 300)] rates_to_inr = {"USD": 83.0, "EUR": 90.0} commission = 0.02 # 2% forex fee budget_inr = 80000 total_inr = 0 category_totals = {} for cat, curr, amt in expenses_foreign: inr_val = (amt * rates_to_inr[curr]) * (1 + commission) total_inr += inr_val category_totals[cat] = category_totals.get(cat, 0) + inr_val max_cat = max(category_totals, key=category_totals.get) print(f"Total Spent in INR: Rs.{total_inr:.2f}") print(f"Largest Expense: {max_cat} (Rs.{category_totals[max_cat]:.2f})") print(f"Budget Status: {'Within Budget' if total_inr <= budget_inr else 'Exceeded'}")
```

Test & Out put:

travel_budget.py - Visual Studio Code

travel_budget.py

```
expenses = [("Hotel", "USD", 400), ("Food", "USD", 150), ("Flight", "EUR", 300)] rates, commission, budget_inr = {"USD": 83.0, "EUR": 90.0}, 0.02, 80000 total_inr, category_totals = 0, {} for cat, curr, amt in expenses: inr_val = (amt * rates[curr]) * (1 + commission) total_inr += inr_val category_totals[cat] = category_totals.get(cat, 0) + inr_val max_cat = max(category_totals, key=category_totals.get) print(f"Total Spent in INR: Rs.{total_inr:.2f} | Largest Expense: {max_cat} (Rs.{category_totals[max_cat]:.2f})")
```

```
TERMINAL - BASH
user@laptop: ~/projects$ python travel_budget.py
Total Spent in INR: Rs.75735.00 | Largest Expense: Hotel (Rs.33864.00)
```

Conclusion: Foreign exchange calculations accurately incorporated 2% conversion fees and confirmed adherence to overall budget caps.

CASE STUDY 10: CAMPUS EVENT FINANCIAL PLANNING

1 PROBLEM STATEMENT & PYTHON IMPLEMENTATION

Calculate event management costs (venue, equipment, catering, certificates), factor in secured corporate sponsorships, and compute the minimum ticket price or registration count required to reach financial break-even point.

```
# Case Study 10: Event Financial Planning venue_cost = 25000 equip_cost = 10000 catering_per_head = 150 cert_per_head = 20 sponsorships = 15000 expected_participants = 200 ticket_price = 250 fixed_costs = venue_cost + equip_cost # 35000 var_cost_per_head = catering_per_head + cert_per_head # 170 net_fixed_cost = fixed_costs - sponsorships # 20000 # Break- even point equation: N * ticket_price = Net_Fixed + N * var_cost # N * ( ticket_price - var_cost) = Net_Fixed contribution_margin = ticket_price - var_cost_per_head # 80 breakeven_count = net_fixed_cost / contribution_margin if contribution_margin > 0 else float('inf') projected_profit = (expected_participants * contribution_margin) - net_fixed_cost print(f"Break-Even Participants Required: {int(breakeven_count)}") print(f"Projected Event Profit: Rs.{projected_profit:.2f}")
```

Test case & Out put:

event_planning.py - Visual Studio Code

event_planning.py

```
venue, equip, cater, cert, sponsor, expected_att, ticket = 25000, 10000, 150, 20, 15000, 200, 250
net_fixed_cost = (venue + equip) - sponsor # Rs. 20,000 contrib_margin = ticket - (cater + cert) # 250 - 170
= Rs. 80 breakeven_count = net_fixed_cost / contrib_margin projected_profit = (expected_att *
contrib_margin) - net_fixed_cost print(f"Break-Even Participants Required: {int(breakeven_count)}")
print(f"Projected Event Profit: Rs.{projected_profit:.2f}")
```

```
TERMINAL - BASH
user@laptop:~/projects$ python event_planning.py
Break-Even Participants Required: 250 | Projected Event Profit: Rs.-4000.00
```

Conclusion: Financial model highlights a 50-participant deficit at expected turnout levels, requiring ticket price adjustments or higher sponsorship targets to prevent deficit.